MATH 485 Assignment#4: Customer Churn Prediction with XGBoost

Part 1: Data Exploration

1. Load the dataset.

```
df = pd.read_csv("telco-customer-churn-by-IBM.csv")
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure
0	7590-VHVEG	Female	0	Yes	No	1
1	5575-GNVDE	Male	0	No	No	34
2	3668-QPYBK	Male	0	No	No	2
3	7795-CFOCW	Male	0	No	No	45
4	9237-HQITU	Female	0	No	No	2

5 rows × 21 columns

2. Report:

○ Number of rows and columns

```
Shape: (7043, 21)
  → 7043 rows  |  21 columns
```

○ List of features

```
Columns: ['customerID', 'gender', 'SeniorCitizen', 'Partner',
'Dependents', 'tenure', 'PhoneService', 'MultipleLines',
'InternetService', 'OnlineSecurity', 'OnlineBackup', 'DeviceProtection',
'TechSupport', 'StreamingTV', 'StreamingMovies', 'Contract',
'PaperlessBilling', 'PaymentMethod', 'MonthlyCharges', 'TotalCharges',
'Churn']
```

○ Target variable

```
Target variable: Churn
```

○ Missing values per column

```
Missing values per column:
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
```

```
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

○ Distribution of the target variable (Churn)

```
Target distribution (raw):
Churn
No     5174
Yes    1869
Name: count, dtype: int64

Target distribution (%):
Churn
No     73.5
Yes    26.5
Name: proportion, dtype: float64
```

3. Briefly describe:

○ What kind of problem this is (classification/regression)

Binary Classification - Customers will churn (yes = 1) or (No = 0).

○ Why this problem is important in a business context

Customer churning is directly linked to revenue. In a business, acquiring new customers usually costs more than retaining existing customers, so it is very important to understand the mood of customers. If they are at risk of leaving the business, companies can take action to retain them by providing discounts, upgrades in the contracts,etc.

Part 2: Data Preprocessing

● Identify numerical and categorical features

```
Shape after cleaning: (7043, 20)
```

customerID column is dropped, because it is not a feature, but a unique identifier.

● Handle missing values appropriately

```
Missing values after conversion:
TotalCharges    11
dtype: int64
```

```
Churn dtype: int64
```

● Convert the target variable into binary form (0/1)

	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines
0	Female	0	Yes	No	1	No	No phone service
1	Male	0	No	No	34	Yes	No
2	Male	0	No	No	2	Yes	No

MonthlyCharges	TotalCharges	Churn
29.85	29.85	0
56.95	1889.50	0
53.85	108.15	1

● Encode categorical variables

```
Categorical features: ['gender', 'SeniorCitizen', 'Partner', 'Dependents',
'PhoneService', 'MultipleLines', 'InternetService', 'OnlineSecurity',
'OnlineBackup', 'DeviceProtection', 'TechSupport', 'StreamingTV',
'StreamingMovies', 'Contract', 'PaperlessBilling', 'PaymentMethod']
```

● Split the data into training and test sets

```
X_train shape: (5634, 19)
X_test  shape: (1409, 19)

Train churn rate: 0.2654
Test  churn rate: 0.2654
```

Part 3: Baseline Model

Train a Logistic Regression model as a baseline.
Evaluate it using:
● Accuracy
● Precision
● Recall
● F1-score
● ROC-AUC

```python
print("=== Baseline: Logistic Regression ===")
print(f"  Accuracy : {accuracy_score(y_test, baseline_preds):.4f}")
print(f"  Precision: {precision_score(y_test, baseline_preds):.4f}")
print(f"  Recall   : {recall_score(y_test, baseline_preds):.4f}")
print(f"  F1-Score : {f1_score(y_test, baseline_preds):.4f}")
print(f"  ROC-AUC  : {roc_auc_score(y_test, baseline_probs):.4f}")
```

```
=== Baseline: Logistic Regression ===
  Accuracy : 0.8041
  Precision: 0.6531
  Recall   : 0.5588
  F1-Score : 0.6023
  ROC-AUC  : 0.8426
```

```python
print("Classification Report — Logistic Regression:")
print(classification_report(y_test, baseline_preds, target_names=['No Churn', 'Churn']))

print("Confusion Matrix:")
print(confusion_matrix(y_test, baseline_preds))
```

```
Classification Report — Logistic Regression:
              precision    recall  f1-score   support

    No Churn       0.85      0.89      0.87      1035
       Churn       0.65      0.56      0.60       374

    accuracy                           0.80      1409
   macro avg       0.75      0.73      0.74      1409
weighted avg       0.80      0.80      0.80      1409

Confusion Matrix:
[[924 111]
 [165 209]]
```

Part 4: XGBoost Model + Hyperparameter Tuning

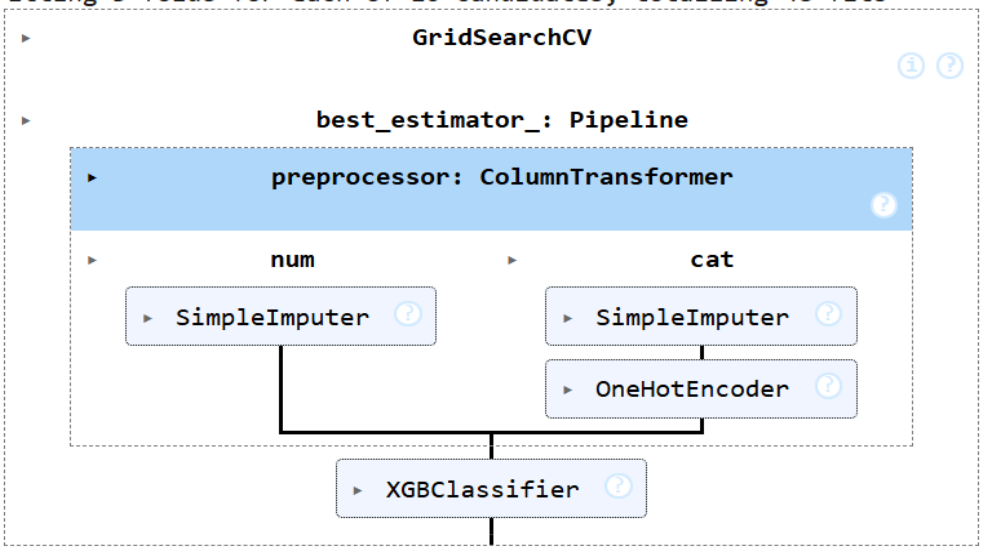
Train an XGBoost classifier and tune at least 3 hyperparameters, such as:

- max_depth
- n_estimators
- learning_rate
- subsample

```
Hyperparameter grid: {'classifier__n_estimators': [100, 200],
'classifier__max_depth': [3, 5], 'classifier__learning_rate': [0.05, 0.1],
'classifier__subsample': [0.8, 1.0]}
Total combinations: 16 × 3-fold CV = 48 fits
```

- Grid search (some example code is provided in the starter notebook), or

```
Fitting 3 folds for each of 16 candidates, totalling 48 fits
```



● Manual tuning
● Clearly state which hyperparameters you tuned :
- n_estimators: number of boosting rounds (trees).
- max_depth: maximum depth of each tree.
- learning_rate (eta): how much each tree contributes.
- subsample: fraction of training rows used per tree.

● Justify your choices briefly

These control the main bias–variance tradeoff in XGBoost:

- n_estimators + learning_rate balance model strength vs overfitting (lower eta usually needs more trees).
- max_depth controls complexity and interactions (deeper trees can overfit).
- Subsample adds randomness to reduce variance and improve generalization.

Part 5: Choose Evaluation Metric

Choose one primary evaluation metric (e.g., F1-score or ROC-AUC).
Explain:
● Why ROC-AUC is appropriate for churn prediction

- Accuracy can look high even if the model is poor. ROC-AUC evaluates how well the model separates churners from non-churners across thresholds, so it stays informative even when classes are imbalanced.
- Companies want the highest-risk customers ranked at the top. ROC-AUC directly measures the probability that the model assigns a higher score to a true churner than to a non-churner, which aligns with how retention teams use predictions.

- The best threshold depends on business constraints (budget, retention offer cost, expected value of saving a customer). ROC-AUC evaluates performance across all thresholds rather than locking you into one cutoff.

● Why it is better than other metrics in this context

- Accuracy can be misleading with imbalance and does not reflect how well the model ranks churn risk.
- F1 depends on choosing a single threshold, and results can change a lot if that threshold is moved. If the business decision threshold is not fixed yet, ROC-AUC is more stable for comparing models during development.
- Precision-only can ignore missed churners, and recall-only can create too many false positives. ROC-AUC summarizes the tradeoff across thresholds, making it a better "overall" metric for model selection.

Part 6: Feature Importance & Interpretation

Using your trained XGBoost model:
● Identify the top 5-10 most important features

```
Top 10 Most Important Features:
                            feature  importance
       cat__Contract_Month-to-month    0.415293
   cat__InternetService_Fiber optic    0.107321
            cat__OnlineSecurity_No    0.066205
               cat__TechSupport_No    0.058200
cat__PaymentMethod_Electronic check    0.036326
                        num__tenure    0.032705
           cat__PaperlessBilling_No    0.024530
           cat__StreamingMovies_Yes    0.024155
             cat__Contract_Two year    0.022521
             cat__Contract_One year    0.018899
```

● Interpret them in plain language
Example:
"Customers with higher monthly charges are more likely to churn…"

- The model indicates that churn risk is associated most strongly by contract structure, followed by service type and whether customers have support related services. Month-to-month contracts dominate feature importance, suggesting customers without long-term commitments are the highest risk group. Billing and payment behavior (electronic check, paperless billing) and tenure also contribute, indicating that customer engagement and relationship length are key factors in retention.

Part 7: Final Comparison & Reflection

Compare:
● Logistic Regression vs XGBoost

Metric	Logistic Regression	XGBoost (Tuned)	Improvement
Accuracy	0.8041	0.8191	+1.50%
Precision	0.6541	0.6760	+2.19%
Recall	0.5561	0.5882	+3.21%
F1-Score	0.6012	0.6291	+2.79%
ROC-AUC	0.8426	0.8672	+2.46%

● Which model performed better
Tuned XGBoost performed better than Logistic Regression on every metric (Accuracy, Precision, Recall, F1, and ROC-AUC). The biggest gain is in Recall and F1, and ROC-AUC improves from 0.8426 to 0.8672.

● Why XGBoost may work better for this dataset :
XGBoost can capture nonlinear patterns and feature interactions (for example, contract type interacting with tenure or services) that Logistic Regression may miss unless you manually engineer interactions.

● One limitation of XGBoost :
It is less interpretable than Logistic Regression and can be more sensitive to hyperparameters, so it needs careful tuning and explainability tools to justify decisions.